

config.py

```
import os
from datetime import timedelta
import sys

# Application Configuration
APP_NAME = "VulnEdu - Educational CVE Analysis Tool"
SECRET_KEY = os.environ.get('SECRET_KEY', 'dev-key-change-in-production')

# Debug environment variables
print("[Config] Environment variables debug:")
db_url = os.environ.get('DATABASE_URL')
print(f"[Config] DATABASE_URL exists: {db_url is not None}")
if db_url:
    print(f"[Config] DATABASE_URL starts with: {db_url[:10]}...")
else:
    print("[Config] WARNING: No DATABASE_URL found")

# Check Render specific variables
print(f"[Config] Running on Render: {'RENDER' in os.environ}")
if 'RENDER' in os.environ:
    print(f"[Config] Render instance type: {os.environ.get('RENDER_SERVICE_TYPE', 'unknown')}")

# NVD API Configuration
NVD_API_URL = "https://services.nvd.nist.gov/rest/json/cves/2.0"
NVD_API_KEY = os.environ.get('NVD_API_KEY', '9d289859-60a3-4f9f-af3c-30fdcddf3918')

# Data Storage Paths
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
DATA_DIR = os.path.join(BASE_DIR, 'data')
CACHE_DIR = os.path.join(DATA_DIR, 'cache')
NVD_DIR = os.path.join(DATA_DIR, 'nvd')
NVD_HISTORICAL_DIR = os.path.join(NVD_DIR, 'historical')
NVD_PROCESSED_DIR = os.path.join(NVD_DIR, 'processed')
CWE_XML_PATH = os.path.join(DATA_DIR, 'CWE_Catalog.xml')

# NVD Data Feeds
NVD_FEEDS_BASE_URL = "https://nvd.nist.gov/feeds/json/cve/1.1"

# Historical years configuration - USE LOCAL FILES
# REDUCED to 3 years for memory efficiency
current_year = 2025
HISTORICAL_YEARS = [current_year - 2, current_year - 1, current_year] # 2023, 2024, 2025

# Database Configuration - CRITICAL: Enable database storage for Render
DATABASE_URL = os.environ.get('DATABASE_URL')
print(f"[Config] Original DATABASE_URL: {DATABASE_URL[:10]} + '...' if DATABASE_URL else 'None'")

# IMPORTANT FIX: Ensure DATABASE_URL is properly processed
if DATABASE_URL and DATABASE_URL.startswith('postgres://'):
    # Fix Render's postgres:// vs postgresql:// URL format issue
    DATABASE_URL = DATABASE_URL.replace('postgres://', 'postgresql://', 1)
    print(f"[Config] Fixed DATABASE_URL format: {DATABASE_URL[:15]}...")

# Only set DATABASE_ENABLED to True if we actually have a valid URL
DATABASE_ENABLED = bool(DATABASE_URL)
print(f"[Config] Database enabled: {DATABASE_ENABLED}")

SQLALCHEMY_DATABASE_URI = DATABASE_URL
SQLALCHEMY_TRACK_MODIFICATIONS = False

# Manually export DATABASE_URL to environment if it wasn't there
if DATABASE_URL and not os.environ.get('DATABASE_URL'):
    print("[Config] Re-exporting DATABASE_URL to environment")
    os.environ['DATABASE_URL'] = DATABASE_URL

# Lazy Loading Configuration - NEW
LAZY_LOAD_ENABLED = True
MAX_MEMORY_USAGE_MB = 450 # Set max memory to less than 512MB to prevent crashes

# Cache Configuration
```

```
CACHE_DURATION = timedelta(minutes=30)
MAX_CACHE_ENTRIES = 10 # Reduced from 100 to save memory

# Data Loading Strategy
RECENT_DAYS_THRESHOLD = 30
USE_LOCAL_FEEDS = True # CHANGED: Use local files
AUTO_UPDATE_FEEDS = False
USE_GITHUB_DATA = False # CHANGED: Disable GitHub

# API Rate Limiting
API_REQUESTS_PER_30_SECONDS = 50
API_TIMEOUT = 30

# Pagination
DEFAULT_PAGE_SIZE = 25
MAX_PAGE_SIZE = 100

# Page-specific data loading flags
LOAD_ALL_DATA_FOR_LEARN_PAGES = False # NEW: Don't load all data for educational pages

# Ensure directories exist
for directory in [CACHE_DIR, NVD_DIR, NVD_HISTORICAL_DIR, NVD_PROCESSED_DIR]:
    os.makedirs(directory, exist_ok=True)

print(f"[Config] Configuration loaded successfully. Database mode: {'ENABLED' if
DATABASE_ENABLED else 'DISABLED'}")
```